Understanding Python Metaprogramming

Writing code that inspects, changes and creates code while the program runs

INTRODUCTION

What is metaprogramming?

- Metaprogramming means writing code that works on other code: it can inspect classes, change how objects behave, or create new classes while the program is running.
- This is possible because in Python everything is an object, including functions and classes.
- A class is itself an object, created by another class called type. A class that creates classes is a metaclass.
- These notes build up step by step: inspect, intercept reading, intercept writing, create classes, add methods, and finally control class creation.

Each arrow means "is created by": type(acc) is Account, type(Account) is type



00_everything_is_an_object.py

```python
class Account:
    pass

acc = Account()

# Every value has a type, including classes themselves
print(type(acc))       # <class '__main__.Account'>
print(type(Account))   # <class 'type'>
print(type(int))       # <class 'type'>

# Classes are ordinary values: store them, pass them around
kinds = [int, str, Account]
print([k.__name__ for k in kinds])   # ['int', 'str', 'Account']
```

RUNTIME OBJECT MODEL

Classes and objects can be inspected at runtime

- Every object remembers the class that created it, in its __class__ attribute.
- Every class records its own name (__name__) and its direct parent classes (__bases__).
- __mro__ (method resolution order) lists every class Python searches, in order, when it looks up an attribute.
- Reading this information while a program runs is called introspection. It is the starting point for all metaprogramming.

01_runtime_object_model.py

```python
class Account:
    pass

class SavingsAccount(Account):
    pass

acc = SavingsAccount()

# __class__: the class that created this object
print(acc.__class__)
# <class '__main__.SavingsAccount'>

# __name__: the class's own name, as text
print(SavingsAccount.__name__)
# SavingsAccount

# __bases__: the classes it directly inherits from
print(SavingsAccount.__bases__)
# (<class '__main__.Account'>,)

# __mro__: the full order Python searches for attributes
print(SavingsAccount.__mro__)
# (<class '__main__.SavingsAccount'>,
#  <class '__main__.Account'>, <class 'object'>)
```

RUNTIME OBJECT MODEL

Why introspection matters

- Introspection lets one function handle objects of any class, without naming those classes in its code.
- Here, describe() uses type() to find the class name and vars() to read the object's fields.
- New classes work with describe() automatically, with no changes to the function.
- Where it is used: JSON serialisers, debuggers and loggers, test frameworks, and ORMs that map classes to database tables.

01b_introspection_in_practice.py

```python
class Patient:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Invoice:
    def __init__(self, number, total):
        self.number = number
        self.total = total

def describe(obj):
    """Works for ANY object: no class names are hard-coded."""
    cls_name = type(obj).__name__
    fields = ", ".join(f"{k}={v!r}" for k, v in vars(obj).items())
    return f"{cls_name}({fields})"

print(describe(Patient("Amina", 34)))
# Patient(name='Amina', age=34)

print(describe(Invoice(101, 250.0)))
# Invoice(number=101, total=250.0)
```

DYNAMIC ATTRIBUTES

Handling missing attributes with __getattr__

- Python calls __getattr__(self, name) only when the normal lookup has failed to find the attribute.
- Attributes that really exist, such as self.values, are found normally, so __getattr__ is never involved.
- This lets an object answer settings.port by looking the name up somewhere else, here in a dictionary.
- If the name cannot be found, raise AttributeError, so the object still behaves like a normal Python object.

02_getattr_dynamic_attributes.py

```python
class Settings:
    def __init__(self, values):
        self.values = values      # a normal attribute

    def __getattr__(self, name):
        # Called ONLY when normal lookup fails
        print(f"  __getattr__ called for {name!r}")
        try:
            return self.values[name]
        except KeyError:
            raise AttributeError(f"No setting called {name!r}") from None

settings = Settings({"debug": True, "port": 8080})

print(settings.port)      # __getattr__ called for 'port', then 8080
print(settings.values)    # found normally: __getattr__ is NOT called

try:
    settings.host         # not in the dict
except AttributeError as err:
    print(err)            # No setting called 'host'
```

DYNAMIC ATTRIBUTES

Why __getattr__ matters: proxy objects

- A proxy is an object that stands in for another object and passes requests on to it.
- Because __getattr__ catches every missing name, the proxy can forward any method without listing them all.
- The proxy can add behaviour on the way, such as logging, permission checks or counting calls.
- Where it is used: configuration objects with dot access, logging and caching wrappers, remote API clients, and lazy loading.

02b_getattr_proxy.py

```
class LoggingProxy:
    """Wraps any object and logs each attribute it passes on."""

    def __init__(self, target):
        self._target = target

    def __getattr__(self, name):
        # Anything the proxy itself lacks is fetched from the target
        print(f"[log] accessing {name}")
        return getattr(self._target, name)

items = LoggingProxy([3, 1, 2])

items.append(5)          # [log] accessing append
items.sort()             # [log] accessing sort
print(items._target)     # [1, 2, 3, 5]
```

NAMESPACE CONTROL

Checking every assignment with __setattr__

- Python calls __setattr__(self, name, value) for every assignment such as p.age = 35, including those inside __init__.
- This makes it the one place to validate or clean values, however and whenever they are set.
- Inside __setattr__, never write self.name = value: that is itself an assignment, so the method would call itself forever.
- Use object.__setattr__(self, name, value) to store the value with Python's normal behaviour.

03_setattr_namespace_control.py

```python
class Patient:
    def __init__(self, name, age):
        self.name = name
        self.age = age            # goes through __setattr__ too

    def __setattr__(self, name, value):
        # Runs for EVERY assignment, e.g. p.age = 35
        if name == "age" and not 0 <= value <= 130:
            raise ValueError(f"age must be 0 to 130, got {value}")
        # Writing self.age = value here would call this method
        # again, forever. object.__setattr__ does the real write.
        object.__setattr__(self, name, value)

p = Patient("Amina", 34)
p.age = 35                        # accepted
print(p.age)                      # 35

try:
    p.age = -5                    # rejected
except ValueError as err:
    print(err)                    # age must be 0 to 130, got -5
```

NAMESPACE CONTROL

Why _setattr_ matters: tracking changes

- Because every assignment passes through __setattr__, the object can record exactly which fields have changed.
- Loading the original values uses object.__setattr__ directly, so it is not counted as a change.
- Only the changed fields then need to be saved, which keeps database updates small and fast.
- Where it is used: ORMs marking fields as “dirty”, read-only (frozen) objects, and keeping screens in sync with data.

03b_setattr_change_tracking.py

```
class Record:
    """Remembers which fields have changed since loading."""

    def __init__(self, **fields):
        # Use object.__setattr__ so loading is not counted as a change
        object.__setattr__(self, "_changed", set())
        for key, value in fields.items():
            object.__setattr__(self, key, value)

    def __setattr__(self, name, value):
        self._changed.add(name)                 # mark as changed
        object.__setattr__(self, name, value)   # then store it

r = Record(name="Amina", ward="B", bed=4)
r.ward = "C"
r.name = "Amina K."

print(sorted(r._changed))   # ['name', 'ward']
# Only these two fields need to be saved to the database
```

DYNAMIC CLASSES

Creating classes with type()

- type(obj) with one argument tells you which class an object belongs to.
- type(name, bases, namespace) with three arguments creates a brand-new class.
- name is the class name; bases is a tuple of parent classes, where () means just object; namespace is a dictionary of attributes and methods.
- This is exactly what a class statement does behind the scenes.

04_dynamic_class_construction_type.py

```python
# 1. type(obj): which class made this object?
print(type(3.14))                    # <class 'float'>

# 2. type(name, bases, namespace): build a NEW class
def greet(self):
    return f"Hello, I am {self.name}"

Person = type(
    "Person",                              # name of the class
    (),                                    # parents: () means object
    {"name": "Ada", "greet": greet},       # attributes and methods
)

# This is exactly what the class statement does for:
# class Person:
#     name = "Ada"
#     def greet(self): return f"Hello, I am {self.name}"

p = Person()
print(p.greet())                     # Hello, I am Ada
print(Person.__name__)               # Person
```

DYNAMIC CLASSES

Why type() matters: classes built from data

- Sometimes the shape of a class is only known when the program runs, for example the columns of a database table.
- A class factory is a function that builds and returns a new class using type().
- Here make_model() creates complete Drug and Ward classes from lists of field names.
- Where it is used: ORMs generating models from tables, dataclasses.make_dataclass, and classes built from JSON schemas.

04b_class_factory.py

```
def make_model(name, fields):
    """Build a simple record class from a list of field names."""

    def __init__(self, **kwargs):
        for field in fields:
            setattr(self, field, kwargs.get(field))

    def __repr__(self):
        values = ", ".join(f"{f}={getattr(self, f)!r}" for f in fields)
        return f"{name}({values})"

    return type(name, (), {"__init__": __init__, "__repr__": __repr__})

# The field names could come from a database table or a JSON schema
Drug = make_model("Drug", ["name", "dose_mg"])
Ward = make_model("Ward", ["code", "beds"])

print(Drug(name="Paracetamol", dose_mg=500))
# Drug(name='Paracetamol', dose_mg=500)
print(Ward(code="B", beds=12))
# Ward(code='B', beds=12)
```

DYNAMIC EXTENSION

Adding methods to a class at runtime

- Functions are objects, so a function can be attached to a class like any other attribute.
- Once attached to the class, it behaves as a normal method: Python passes the object in as self.
- Every instance can use the new method, even instances created before it was added.
- The original class definition is never edited.

05_dynamic_method_binding.py

```
class Report:
    def __init__(self, title):
        self.title = title

early = Report("Q1")            # created BEFORE the method exists

def summary(self):
    return f"Report {self.title} is ready"

# Attach the plain function to the class
Report.summary = summary

print(early.summary())          # Report Q1 is ready
print(Report("Q2").summary())   # Report Q2 is ready
```

DYNAMIC EXTENSION

Why it matters: generating methods

- A loop can create and attach many similar methods from a list of names, instead of writing each by hand.
- Here, one method is generated for each API endpoint, so client.patients(…) and client.invoices(…) appear automatically.
- Adding a new endpoint only means adding a name to the list.
- Where it is used: API client libraries, plugin systems, and replacing methods temporarily in tests (monkey-patching).

05b_generated_api_methods.py

```python
class ApiClient:
    base_url = "https://api.example.com"

def make_endpoint(path):
    """Create one method that calls a single API endpoint."""
    def call(self, **params):
        query = "&".join(f"{k}={v}" for k, v in params.items())
        return f"GET {self.base_url}/{path}?{query}"
    call.__name__ = path
    return call

# One method per endpoint, generated from a list of names
for path in ["patients", "appointments", "invoices"]:
    setattr(ApiClient, path, make_endpoint(path))

client = ApiClient()
print(client.patients(ward="B"))
# GET https://api.example.com/patients?ward=B
print(client.invoices(month=10))
# GET https://api.example.com/invoices?month=10
```

METACLASSES

Controlling class creation with a metaclass

- A metaclass is a class whose instances are classes. The default metaclass is type.
- To write one, subclass type and override __new__, which runs while each class is being created.
- At that moment the class body is still a plain dictionary (namespace), so it can be checked or changed.
- Here, any subclass without a run() method is rejected as soon as its class statement runs, long before it is used.

06_metaclass_interface_verifier.py

```python
class RequireRun(type):
    """A metaclass: it controls how CLASSES are created."""

    def __new__(mcls, name, bases, namespace):
        # Runs when a class statement is executed,
        # before the new class object exists
        if bases and "run" not in namespace:
            raise TypeError(f"{name} must define run()")
        return super().__new__(mcls, name, bases, namespace)

class Base(metaclass=RequireRun):   # no parents: allowed
    pass

class Job(Base):                    # defines run(): allowed
    def run(self):
        return "ok"

try:
    class BrokenJob(Base):          # no run(): rejected at once
        pass
except TypeError as err:
    print(err)                      # BrokenJob must define run()

print(type(Job))                    # <class '__main__.RequireRun'>
```

METACLASSES

Why metaclasses matter: automatic registration

- A metaclass sees every class that uses it, at the moment each class is defined.
- Here, every subclass of Plugin is added to a registry automatically; nobody has to remember to register it.
- The program can then choose a plugin by name, for example from a user setting.
- Where it is used: Django and SQLAlchemy models, and Python's own abc.ABCMeta and enum.EnumMeta.

06b_metaclass_plugin_registry.py

```python
class PluginMeta(type):
    """Records every plugin class the moment it is defined."""
    registry = {}

    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if bases:                               # skip the base class
            mcls.registry[name.lower()] = cls
        return cls

class Plugin(metaclass=PluginMeta):
    pass

class CsvExporter(Plugin):
    def export(self):
        return "saved as CSV"

class PdfExporter(Plugin):
    def export(self):
        return "saved as PDF"

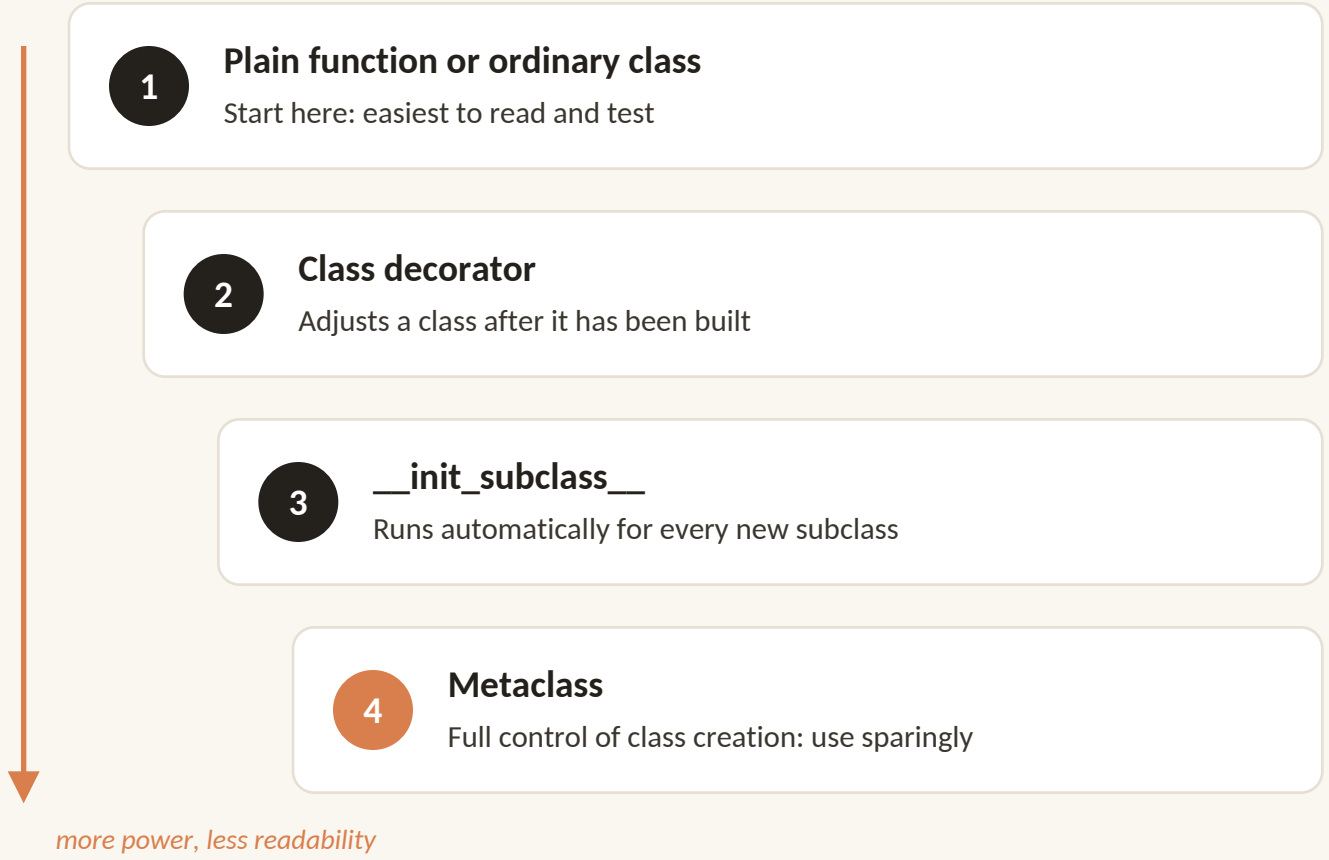
print(list(PluginMeta.registry))       # ['csvexporter', 'pdfexporter']

choice = "pdfexporter"                 # e.g. read from user settings
print(PluginMeta.registry[choice]().export())   # saved as PDF
```

DESIGN PRINCIPLES

Use the simplest tool that works

- Rule of least power: choose the simplest technique that solves the problem.
- Each step up the ladder gives more control, but makes the code harder to read, test and debug.
- Most class checks and registrations can be done with __init_subclass__ or a class decorator.
- Keep a metaclass for jobs that must change the class body before the class exists.
- A class can have only one metaclass, so two libraries that each use their own metaclass can clash.



DESIGN PRINCIPLES

Simpler alternatives to a metaclass

- __init_subclass__ is a normal class method that Python calls for every new subclass.
- It can perform the same run() check as the earlier metaclass, with no metaclass at all.
- A class decorator is a function that receives a finished class and can add to it or check it.
- Both are easier to read than a metaclass, and both work alongside other libraries' metaclasses.

07_simpler_alternatives.py

```
# Alternative 1: __init_subclass__ runs for every new subclass
class Base:
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        if "run" not in cls.__dict__:
            raise TypeError(f"{cls.__name__} must define run()")

class Job(Base):
    def run(self):
        return "ok"

try:
    class BrokenJob(Base):
        pass
except TypeError as err:
    print(err)                # BrokenJob must define run()

# Alternative 2: a class decorator runs AFTER the class is built
def tag(cls):
    cls.meta_tag = True
    return cls

@tag
class Widget:
    pass

print(Widget.meta_tag)        # True
```

SUMMARY

Choosing the right technique

Technique	What it does	Typical use
`type(), vars(), __mro__`	Inspects objects and classes at runtime	Serialisers, debuggers, generic helpers
`__getattr__`	Supplies attributes that normal lookup cannot find	Configuration objects, proxies, API clients
`__setattr__`	Checks or records every assignment	Validation, change tracking, read-only objects
`type(name, bases, ns)`	Creates a new class while the program runs	Models generated from tables or schemas
`Class.method = func`	Adds methods to an existing class	Plugins, generated API methods, test patches
`__init_subclass__ / decorator`	Checks or adjusts classes as they are defined	Most validation and registration needs
`Metaclass`	Controls how classes themselves are built	Frameworks such as Django and SQLAlchemy

ACTIVITIES OVERVIEW

6 Activities to Consolidate Concepts

1

Dynamic Metadata Introspection

Write a function that prints any class's name, parents and MRO.

2

Dynamic Configuration Wrapper

Give a dictionary dot-notation access (config.port) using __getattr__.

3

Strict Attribute Validation

Use __setattr__ to reject values of the wrong type or range.

4

Dynamic Class Factory

Build record classes from lists of field names using type().

5

Dynamic API Method Generator

Attach one method per endpoint name to an API client class.

6

Metaclass Interface Verifier

Reject classes that lack required methods or a docstring when they are defined.